

Augmented Vision: FoodLens AR

Abschlussbericht

Abschlussbericht von David Bassauer, Kartikeya Chaudhary, Weiyang Chen

Tag der Einreichung: 23.09.2024

Gutachter: Pavel Rojtberg, Felix Gorschlüter
Darmstadt



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Augmented Vision



Inhaltsverzeichnis

1	Projektidee	3
2	Zwischenbericht 1	5
3	Zwischenbericht 2	9
4	Abschlussbericht	12

1 Projektidee

Dieses Kapitel beinhaltet eine kurze Zusammenfassung unserer Projektidee, bei der die Vision, Teilziele und die zu verwendenden Technologien aufgezählt werden.

Vision

Die Vision dieses Projekts ist es, eine Augmented Reality Anwendung zu entwickeln, welche die Kameraaufnahme von Lebensmitteln oder Zutaten ermöglicht, sodass aus den mittels Algorithmen erkannten Zutaten Rezepte generiert werden können.

Teilziele

Das erste Teilziel besteht in der Detektion von Lebensmitteln. Nachdem die Lebensmittel erkannt wurden, sollen sie benannt werden. Anschließend werden Rezepte erstellt, die auf den erkannten Lebensmitteln basieren. Eine zusätzliche Idee, die jedoch aufgrund möglicher Kosten nur vorerst betrachtet wird, ist die Verbindung mit einem Large Language Model, um kreative Rezepte zu erhalten.

Technologien

Für die Umsetzung dieses Projekts sollen verschiedene Technologien zum Einsatz kommen. OpenCV wird für die Bildverarbeitung verwendet. Python dient als Programmiersprache zur Implementierung der verschiedenen Schritte. Die Bilder der Lebensmittel können entweder aus einer bestehenden Datenbank stammen oder über eine Webcamanbindung erfasst werden.

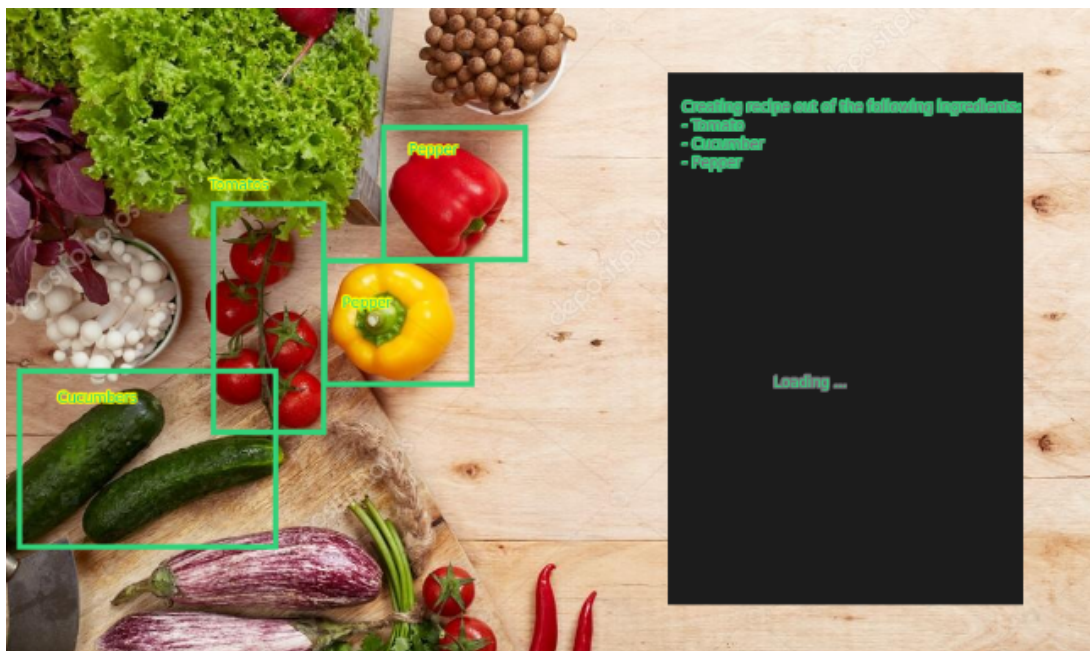


Abbildung 1.1: Projektvision

2 Zwischenbericht 1

Dieses Kapitel beinhaltet unsere bisherigen Fortschritte und Hürden, die bei der Entwicklung unserer Projekt-idee aufgetreten sind. Außerdem wird am Ende des Kapitels noch kurz auf die nächsten Schritte hingewiesen.

Objekterkennung durch Templatematching

Der erste Versuch zur Objekterkennung wurde mit Template Matching mithilfe von zuvor aufgenommenen Bildern durchgeführt. Als Input für die Erkennung wurde ein Video verwendet. Dabei traten jedoch mehrere Probleme auf.

Template Matching funktioniert sehr schlecht, wenn das zu erkennende Template nicht exakt im Inputmaterial zu erkennen ist. Dies führte zu Problemen beim Matching-Prozess selbst, was bedeutet dass das Template nicht im Inputmaterial gefunden werden konnte.

Ein weiteres Problem war, dass ein Video als Input verwendet wurde. Die Größe des zu erkennenden Objekts stimmte aufgrund von verschiedenen Perspektiven der Aufnahme oft nicht mit dem Template überein. Um dieses Problem zu lösen, wurde eine "Pyramide" mit verschiedenen Auflösungen des zu erkennenden Templates erstellt, sodass eine größere Bandbreite an möglichen Template-Größen abgedeckt werden konnte. Trotz dieser Maßnahmen funktionierte die Erkennung nicht gut. Objekte wurden nicht zuverlässig erkannt. Man könnte meinen, dass es funktioniert, wenn die Bounding Box groß genug eingestellt wird, aber in der Realität lief die Erkennung sehr schlecht.



Abbildung 2.1: Problematisches Templatematching

Objekterkennung durch Feature Matching

Daraufhin wurde auf Feature Matching umgestellt. Zunächst wurde ein Feature Matching-Prozess mithilfe von OpenCV und Python implementiert. Hierfür wurde ein ORB-Detektor verwendet.

Ein ORB-Detektor (Oriented FAST and Rotated BRIEF) ist ein Algorithmus zur schnellen und robusten Merkmalerkennung, der insbesondere für Echtzeitanwendungen geeignet ist. Er kombiniert die Vorteile der FAST-Schlüsselpunkterkennung und des BRIEF-Descriptors, um skalierungs- und rotationsinvariante Merkmale effizient zu identifizieren.

Als Input wurde hier ebenfalls ein Video verwendet. Als Referenzbilder wurden Fotos des Objekts aus verschiedenen Perspektiven verwendet, sodass die besten Features während der Erkennung aus allen Referenzbildern

ausgewählt und verwendet wurden. Das Ergebnis sah zunächst vielversprechend aus, wie in der folgenden Abbildung zu erkennen ist. Die Features werden sehr gut gematcht und das Objekt wird dementsprechend erkannt. Auch hier kommt es aufgrund des Inputvideos zur Veränderung der Perspektive, was dazu führt, dass das Zeichnen der Bounding Box schwierig ist. Aus diesem Grund haben wir die Dimensionen der Bounding Box zunächst als fix festgelegt.

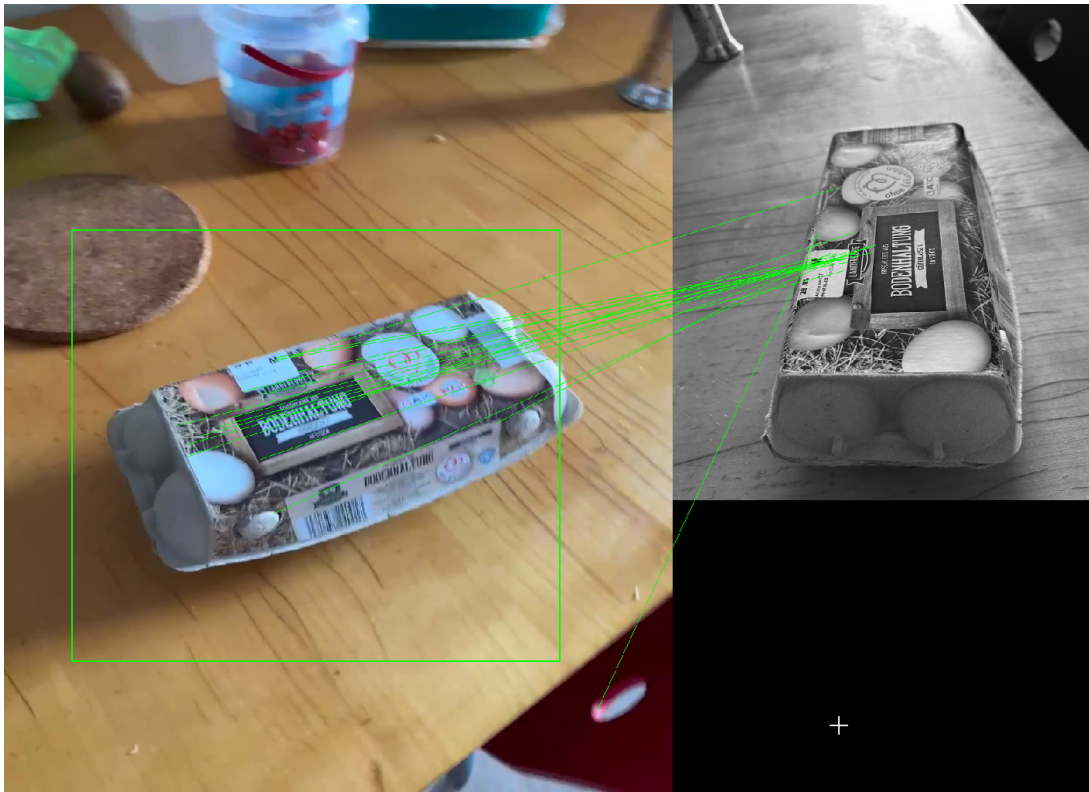


Abbildung 2.2: Feature Matching mit einem Objekt

Der nächste Schritt war, den Prozess der Feature Detection auf mehrere Objekte im Input-Video anzuwenden. Hier traten jedoch große Probleme auf, da plötzlich die verschiedenen Features falsch erkannt wurden. Die Visualisierung des Feature Matchings zeigte, dass viele Features an Stellen im Input-Video erkannt wurden, die normalerweise nicht dazu passen sollten. Beispielsweise wurden von dem Eierkarton Features am Stuhl oder an anderen Objekten erkannt.

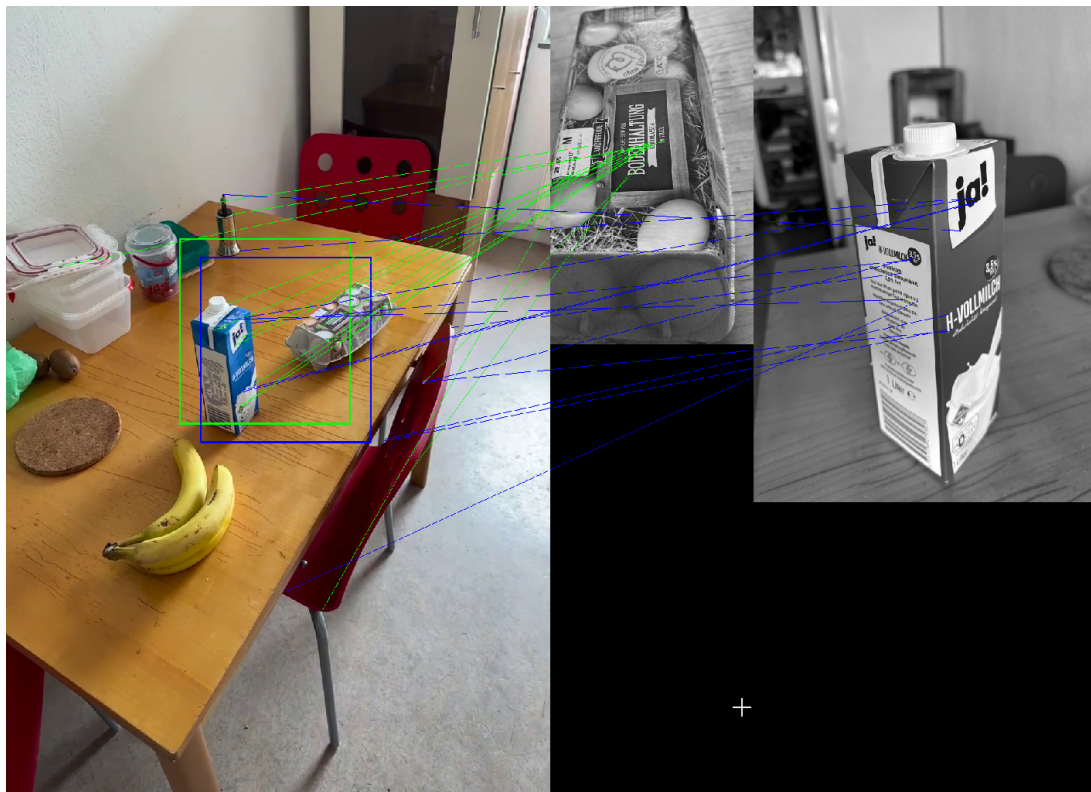


Abbildung 2.3: Feature Matching mit zwei Objekten

Nächste Schritte

Um eine Objekterkennung von mehreren Objekten zu gewährleisten, muss die Implementierung des Feature Matchings mit mehreren Objekten verbessert werden. Die nächsten Schritte bestehen also daraus, die Probleme genauer zu untersuchen und zu lösen. Wenn dieses Problem nicht gelöst werden sollte, möchten wir uns offen halten, nach anderen Methoden zu suchen, um unsere Vision in Realität umzusetzen. Außerdem muss noch eine grafische Benutzeroberfläche für die Rezeptgenerierung implementiert werden. Für die Benutzeroberfläche ist die Idee, dass wir ein Fenster erstellen, welches das Video mit den Bounding Boxes der Objekterkennung anzeigt und daneben ein Feld dargestellt wird, welches die Informationen zu den erkannten Zutaten und das generierte Rezept beinhaltet.

3 Zwischenbericht 2

Dies Kapitel beinhaltet eine kurze Zusammenfassung unserer Ergebnisse für den Zwischenbericht 2.

Verbesserung des Feature Matching

Im Zwischenbericht 1 haben wir für das Feature Matching einen ORB-Detektor verwendet, welche die Merkmale unserer Objekte im Video finden sollte. Die Ergebnisse waren nicht zufriedenstellend, also haben wir an unserem Algorithmus gearbeitet und haben damit begonnen die Art des Detektors zu verändern.

Im Zuge der weiteren Entwicklung haben wir uns für einen SIFT-Detektor (Scale-Invariant Feature Transform) entschieden. Dieser ist ebenfalls, wie auch ORB, ein Algorithmus zur Erkennung und Beschreibung von Merkmalen in Bildern. Diese Merkmale können verwendet werden, um Objekte in Bildern zu erkennen, Bilder zu vergleichen und Szenen zu rekonstruieren. Im Vergleich zum ORB-Algorithmus zeichnet sich SIFT durch eine höhere Robustheit aus. Er ist invariant gegenüber Rotation und Skalierung, sodass die Ergebnisse hierdurch besser werden. Außerdem ist SIFT auch weniger anfällig für Beleuchtungsunterschiede und verwendet mehr Diskriptoren als ORB. Für diese Verbesserung der erkannten Matches, büßt SIFT einen Teil der Schnelligkeit ein, was aber in diesem Projekt nicht von großer Bedeutung ist.

Mit der Verwendung von SIFT ist es nun gelungen mehrere Objekte aufeinmal in einem Input zu erkennen. Ein Fehler, den wir in Zwischenbericht 1 gemacht haben, waren die Referenzfotos, die wir verwendet haben. Diese waren aus verschiedenen Perspektiven und beinhalteten Teile von Hintergründen, die im Featurematching gar nicht beachtet werden sollten. In der folgenden Abbildung ist eines der Testreferenzfotos zu erkennen. Wir haben hier keine Lebensmittelprodukte verwendet, da gerade keine in greifbarer Nähe waren.



Abbildung 3.1: Referenzfoto für Feature Matching

Die Verbesserung der erkannten Matches kann, ähnlich wie in Zwischenbericht 1, erkannt werden. Es gibt immernoch kleinere Ausreißer, jedoch wird das Objekt sehr gut erkannt.

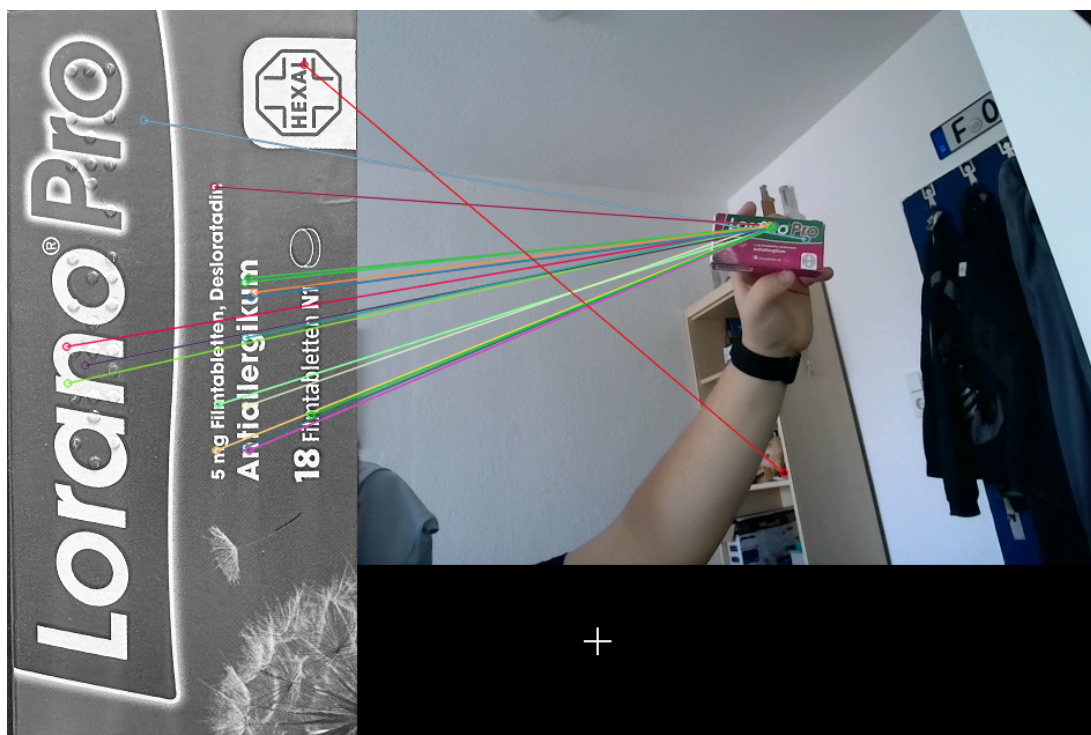


Abbildung 3.2: Feature Matching Visualisierung mit SIFT

Abschließend möchten wir noch eine letzte Abbildung präsentieren, welche zeigt, dass die Erkennung von mehreren Objekten im selben Bild zu erkennen ist.

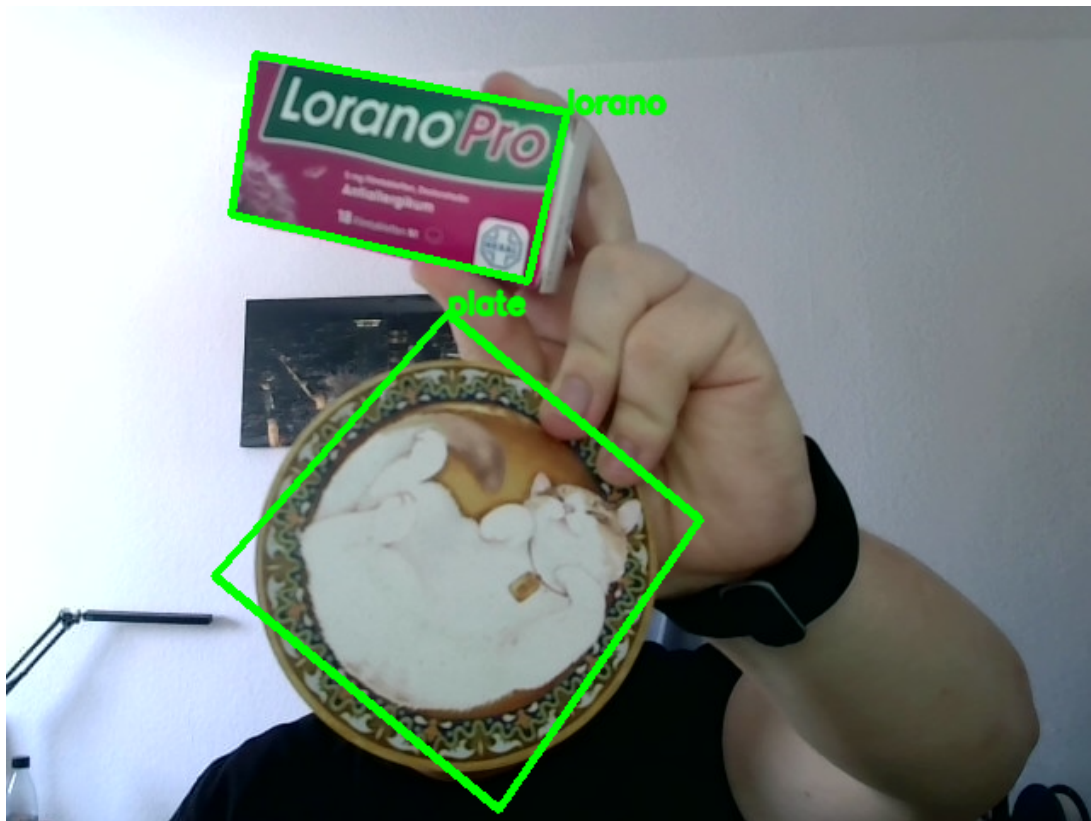


Abbildung 3.3: Erkennung von mehreren Objekten gleichzeitig

Finale Schritte

Als finale Schritte für den Abschluss dieses Projekts sind noch ein paar Dinge zu erledigen. Der Threshold für die Anzahl gefundener Matches muss noch so angepasst werden, dass keine zufällige Erkennung von nicht vorhandenen Objekten im Bild geschehen kann. Außerdem möchten wir noch eine einfache GUI für die Realisierung unserer Projektidee implementieren. Diese GUI soll anzeigen, welche Objekte erkannt wurden und auf Basis dieser erkannten Objekte ein Rezept ausgeben, welches mit den erkannten Lebensmittelprodukten zubereitet werden kann.

4 Abschlussbericht

Dieses Kapitel stellt eine Zusammenfassung unserer Ergebnisse für die finale Abgabe dar.

Projektidee

Die Projektidee beinhaltet das Erstellen einer Applikation zur Lebensmitteldetektion und der Generierung von Rezepten anhand der erkannten Lebensmittel. Hierfür soll mit einer Kamera eine Menge an Lebensmitteln gescannt werden und anschließend automatisch ein Rezept generiert werden, welches mit den erkannten Lebensmitteln gekocht werden kann.

Entwicklung und Problemlösung

Ursprünglich war die Idee, die Objektdetektion mit Hilfe von Templatematching zu ermöglichen. Hierfür haben wir eine Bildpyramide aus den Referenzbildern erzeugt, was ermöglichen würde, Objekte in verschiedenen Auflösungen zu detektieren. Dieses Verfahren hat sich nicht als funktionsfähig für unsere Anwendung herausgestellt, sodass wir zu Feature Matching umgestiegen sind.

Zunächst haben wir mit Hilfe von OpenCV und Python einen ORB-Detektor verwendet, welcher für Objektdetektion verantwortlich war (Bilder zu diesem Verfahren, finden Sie in den vorhergehenden Kapiteln). Jedoch waren unsere Referenzbilder schlecht, da wir sie lediglich aus verschiedenen Perspektiven fotografiert haben. Dies führte zu inkonsistenter Erkennung und fehlerhaften Matches, vor allem bei der Detektion von mehreren Objekten in einem Bild.

Um mehrere Objekte erkennen zu können und stabilere Ergebnisse zu erhalten, haben wir im nächsten Schritt anstatt des ORB-Algorithmus, den SIFT-Algorithmus verwendet. Zudem haben wir nun hochauflösende Referenzfotos gemacht und sie so zugeschnitten, sodass nur Features vom Objekt selbst zu erkennen sind. Dies ermöglichte uns eine stabilere Erkennung und auch gute Ergebnisse beim Erkennen von mehreren Objekten auf einmal.

An diesem Punkt waren unsere Ergebnisse fast zufriedenstellend. Wir haben jedoch bemerkt, dass wir oft Matches erhalten von Objekten, die gar nicht im aktuellen Bild zu sehen sind. Neben der Anpassung des Matches-Schwellenwertes haben wir uns einen weiteren Weg überlegt, dieses Problem zu umgehen. Uns ist aufgefallen, dass zwar viele Matches gefunden werden, aber keine Bounding-Box zu diesen Matches gezeichnet wird. Dies nutzten wir also als Möglichkeit, einen weiteren Filter für unsere Matchingergebnisse einzubauen. Wir haben alle erkannten Matches, die eine sehr kleine Bounding Box haben, ausgefiltert, sodass diese fehlerhaften Matches nicht mehr berücksichtigt wurden. Nach diesen Änderungen fehlte nur noch eine

einfache GUI, welche anhand der Objekte ein Rezept generiert. Wir haben für verschiedene Kombinationen von erkannten Objekten demnach Rezepte erstellt, die je nach erkannten Lebensmitteln ausgegeben wird.

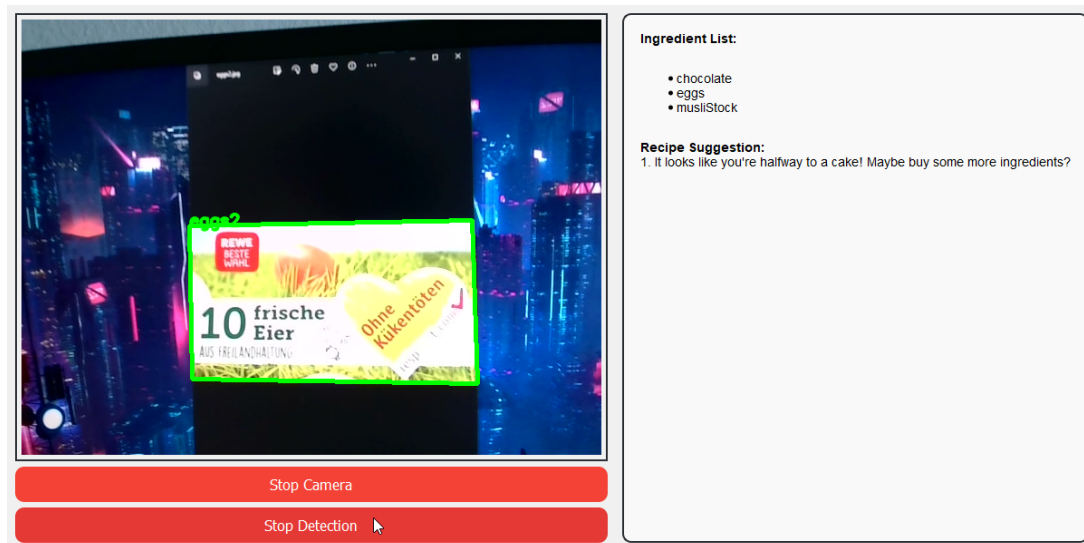


Abbildung 4.1: Finale GUI von FoodLensAR

Zusammenfassung und Ausblick

Zusammenfassend können wir sagen, dass wir unserer Projektvision bereits sehr nahe gekommen sind. Mit FoodLensAR ist es möglich, Lebensmittel bzw. Objekte mit Hilfe von Referenzfotos zu detektieren und ein Rezept für die ermittelten Lebensmittel ausgeben zu lassen. Da nach aktuellem Stand die Rezeptgenerierung hard-coded ist, könnte man die Applikation um eine Anbindung an eine LLM erweitern, sodass eine automatische Rezeptgenerierung ermöglicht wird.